



Tecnológico de Monterrey

**Instituto Tecnológico y de Estudios Superiores de Monterrey.
Campus Estado de México**

Inteligencia artificial avanzada para la ciencia de datos II
TC3007C | (Gpo 501)

Deep Learning for Image Classification

Portafolio Implementación - Módulo 2

Profesor

Julio Guillermo Arriaga Blumenkron

Estudiante

Ulises Jaramillo Portilla..... | A01798380

Fecha de entrega: 11 de noviembre del 2025

1. Introducción.....	3
1.1. Objetivo.....	3
2. Planteamiento del problema.....	3
2.1. Problema a resolver.....	3
3. Conjunto de datos y preprocesamiento.....	4
3.1. Dataset.....	4
3.2. Descarga y organización.....	4
3.3. Transformaciones y Data Augmentation.....	4
3.3.1. Preprocesamiento.....	5
3.3.2. Aumento de datos (Data Augmentation).....	5
4. Desarrollo del modelo.....	5
5. Entrenamiento del modelo.....	6
5.1. Fase 1 – Modelo congelado.....	6
5.1.1. Configuración utilizada:.....	6
5.1.2. Resultado obtenido:.....	6
5.2. Fase 2 – Fine-tuning.....	6
5.2.1. Configuración utilizada:.....	6
5.2.2. Resultado obtenido:.....	7
6. Ajuste del modelo y regularización.....	7
6.1. Dropout.....	7
6.2. Regularización implícita mediante Transfer Learning.....	7
6.3. Fine-tuning parcial de capas profundas.....	7
6.4. Programación dinámica de la tasa de aprendizaje (ReduceLROnPlateau).....	7
6.5. Selección automática del mejor modelo (ModelCheckpoint).....	8
7. Resultados y evaluación.....	8
7.1. Desempeño General en el Conjunto de Prueba.....	8
7.2. Classification report.....	8
7.2.1. Interpretación de los resultados.....	9
7.3 Matriz de confusión.....	9
7.3.1. Análisis de confusiones más relevantes.....	9
7.3.2. Conclusión del análisis de la matriz.....	10
8. Gráficas de entrenamiento.....	10
8.1. Fase 1 – Modelo Congelado.....	10
8.1.1. Accuracy.....	11
8.1.2. Loss.....	11
8.1.3. Conclusión de la Fase 1.....	11
8.2. Fase 2 – Fine-tuning del Modelo.....	11
8.2.1. Accuracy.....	12
8.2.2. Loss.....	12
8.2.3. Conclusión de la Fase 2.....	12
8.3. Conclusión General sobre las Gráficas.....	12
9. Aplicación: Función para probar imágenes nuevas.....	13
9.1. Conclusión de la funcionalidad.....	14
10. Conclusiones.....	14

1. Introducción

En los últimos años, el deep learning se ha consolidado como una de las herramientas más poderosas para el procesamiento y análisis de imágenes, permitiendo automatizar tareas que antes requerían intervención humana extensa. Gracias a arquitecturas modernas basadas en convoluciones, hoy es posible construir modelos capaces de identificar patrones visuales complejos y generalizar a nuevos datos de manera eficiente.

En este proyecto se implementa un sistema completo de clasificación de escenas utilizando TensorFlow y Keras, siguiendo un pipeline integral que abarca: adquisición y preparación del dataset, diseño del modelo, entrenamiento en múltiples fases, evaluación con métricas cuantitativas y cualitativas, y finalmente la inferencia con imágenes externas no vistas durante el entrenamiento. Para ello se emplea el dataset *Intel Image Classification*, compuesto por seis categorías de escenas naturales y urbanas. Este conjunto de datos es considerado realista debido a su variabilidad en iluminación, ángulos, clima y ruido visual, lo que permite evaluar adecuadamente la capacidad de generalización del modelo.

1.1. Objetivo

Desarrollar e implementar un modelo de deep learning basado en técnicas modernas de visión por computadora, capaz de clasificar imágenes de escenas en seis categorías distintas. El objetivo es aplicar de manera correcta y efectiva los conceptos vistos en el módulo, incluyendo el uso de arquitecturas preentrenadas, regularización, fine-tuning, manejo de datasets reales, y generación de predicciones con imágenes nuevas.

Al finalizar, se busca obtener un modelo robusto, evaluado con métricas estándar, y documentar el proceso completo de diseño, implementación y análisis de resultados.

2. Planteamiento del problema

En diversos entornos como plataformas de turismo, sistemas de monitoreo ambiental, fotografía móvil, clasificación automática de contenido o análisis satelital existe la necesidad de procesar grandes volúmenes de imágenes de manera rápida y precisa. Realizar esta tarea manualmente implica un trabajo considerable, además de ser un proceso subjetivo, costoso y poco escalable.

La diversidad visual en las escenas hace que la clasificación sea un desafío técnico relevante: distintos ángulos, condiciones de iluminación, variaciones climáticas o ruido visual pueden dificultar la identificación humana y computacional. Ante ello, se requiere un sistema automatizado que pueda aprender patrones representativos y realizar predicciones consistentes en escenarios reales.

2.1. Problema a resolver

Diseñar y entrenar un modelo de deep learning capaz de clasificar correctamente imágenes de diferentes tipos de escenas, con el fin de automatizar procesos basados en información visual

y reducir la carga manual, aumentando la eficiencia y precisión en la organización y análisis de grandes volúmenes de imágenes.

3. Conjunto de datos y preprocesamiento

3.1. Dataset

Para este proyecto se utilizó el dataset *Intel Image Classification*, el cual contiene más de 25,000 imágenes organizadas en seis categorías de escenas:

- **buildings**
- **forest**
- **glacier**
- **mountain**
- **sea**
- **street**

Este conjunto de datos presenta condiciones visuales altamente variables, incluyendo diferencias en iluminación, clima, ángulos de captura y presencia de ruido. Dichas variaciones lo convierten en un dataset adecuado para evaluar la capacidad de generalización de modelos modernos, evitando las limitaciones de los llamados *toy datasets*.

3.2. Descarga y organización

El dataset fue descargado directamente desde Kaggle utilizando la API oficial, garantizando reproducibilidad y evitando descargas manuales. Una vez obtenido, se reestructuró en los siguientes subconjuntos:

- **Entrenamiento (80%):** 11,224 imágenes
- **Validación (20%):** 2,810 imágenes
- **Prueba (test original del dataset):** 3,000 imágenes

La división del conjunto de entrenamiento en entrenamiento y validación se realizó mediante una partición aleatoria controlada con una semilla fija, asegurando resultados consistentes en posteriores ejecuciones.

3.3. Transformaciones y Data Augmentation

Antes del entrenamiento, las imágenes pasaron por un proceso de preprocesamiento y aumento de datos con el fin de mejorar la calidad del aprendizaje y reducir el riesgo de sobreajuste. Las transformaciones aplicadas fueron:

3.3.1. Preprocesamiento

- **Redimensionamiento** de todas las imágenes a **224 × 224 píxeles**, tamaño estándar para EfficientNet.
- **Normalización** del contenido visual mediante la función oficial de preprocesamiento de EfficientNet, que ajusta los valores al rango esperado por el modelo base.

3.3.2. Aumento de datos (Data Augmentation)

- RandomFlip horizontal
- RandomRotation (0.1)
- RandomZoom (0.1)

Estas técnicas incrementan la diversidad del conjunto de entrenamiento sin necesidad de agregar imágenes adicionales, permitiendo que el modelo aprenda patrones más robustos y generalice adecuadamente frente a condiciones reales de captura.

4. Desarrollo del modelo

Para resolver la tarea de clasificación de escenas se empleó la arquitectura **EfficientNetB0**, un modelo moderno que combina convoluciones compuestas, escalado eficiente y un excelente balance entre precisión y uso de recursos. Esta arquitectura ha demostrado un rendimiento competitivo en múltiples benchmarks, por lo que resulta adecuada para transfer learning en datasets de tamaño medio como el utilizado en este proyecto.

El modelo final se construyó siguiendo el siguiente flujo:

1. **Capa de entrada:** Con imágenes de tamaño 224×224×3.
2. **Pipeline de data augmentation:** Encargado de generar variaciones realistas durante el entrenamiento.
3. **EfficientNetB0 preentrenada en ImageNet:** Cargada sin su capa fully-connected para reutilizar únicamente las representaciones convolucionales.
4. **GlobalAveragePooling2D:** Transforma los mapas de características en un vector compacto.
5. **Capa Dropout (0.3):** Reducir el riesgo de sobreajuste.
6. **Capa densa de 256 unidades (ReLU):** Aprendizaje de patrones de alto nivel.
7. **Otra capa Dropout (0.3):** Reforzar la regularización.
8. **Capa final Softmax:** Con 6 neuronas, una por cada clase del dataset.

Esta combinación permite obtener un modelo ligero, escalable y capaz de aprovechar eficientemente las características generadas por la red base preentrenada.

5. Entrenamiento del modelo

El proceso de entrenamiento se realizó en dos fases, siguiendo la metodología estándar para transfer learning en arquitecturas preentrenadas. Este enfoque permite estabilizar el entrenamiento inicial y posteriormente ajustar las capas más profundas para mejorar el desempeño.

5.1. Fase 1 – Modelo congelado

En esta primera etapa, todas las capas de EfficientNetB0 permanecieron congeladas y únicamente se entrenaron las capas densas añadidas al final del modelo. Esto evita que los pesos preentrenados sufran modificaciones abruptas y permite que la red se adapte gradualmente al dominio del dataset.

5.1.1. Configuración utilizada:

- **Optimizador:** Adam (learning rate = $1e-3$)
- **Callbacks:**
 - ModelCheckpoint (guardar el mejor modelo)
 - ReduceLROnPlateau (ajuste dinámico de la tasa de aprendizaje)
 - EarlyStopping (detener entrenamiento si no hay mejora)

5.1.2. Resultado obtenido:

- **Exactitud en validación** ≈ 0.93

Esta fase permitió estabilizar rápidamente las nuevas capas sin sobreentrenar la red.

5.2. Fase 2 – Fine-tuning

Finalizada la primera etapa, se procedió a realizar *fine-tuning* descongelando aproximadamente las últimas **20 capas** convolucionales de EfficientNetB0. Esto permite ajustar representaciones más profundas y especializarlas al dataset de escenas.

5.2.1. Configuración utilizada:

- **Optimizador:** Adam (learning rate = $1e-4$)
- **Capas entrenables:** últimas ≈ 20 capas del modelo base
- **Callbacks:** mismos de la fase anterior

5.2.2. Resultado obtenido:

- `val_accuracy` ≈ 0.937

El fine-tuning logró un incremento moderado pero consistente en el desempeño del modelo, sin evidenciar sobreajuste ni degradación de las curvas de validación.

6. Ajuste del modelo y regularización

Durante el desarrollo del modelo se implementaron diversas estrategias de ajuste y regularización con el objetivo de mejorar la estabilidad del entrenamiento, reducir el riesgo de sobreajuste y optimizar el desempeño final en el conjunto de validación y prueba. Estas técnicas son fundamentales al trabajar con arquitecturas profundas y datasets visuales con alta variabilidad.

Las principales mejoras aplicadas fueron las siguientes:

6.1. Dropout

Se añadieron capas *Dropout* con una tasa del 30% en las etapas finales del modelo. Esta técnica apaga aleatoriamente un porcentaje de neuronas durante el entrenamiento, evitando que la red memorice patrones específicos del conjunto de entrenamiento y promoviendo una mejor generalización.

6.2. Regularización implícita mediante Transfer Learning

El uso de EfficientNetB0 preentrenada en ImageNet aporta una base sólida de características visuales ya optimizadas. Esta transferencia de conocimiento actúa como una forma de regularización natural, ya que el modelo parte de parámetros robustos en lugar de entrenamiento desde cero.

6.3. Fine-tuning parcial de capas profundas

Después de estabilizar las capas densas, se descongelaron las últimas ≈ 20 capas convolucionales de la red base para ajustar representaciones profundas específicas del dominio del dataset. Este procedimiento permite refinar detalles sin sobreajustar las primeras capas, responsables de la extracción de características más generales.

6.4. Programación dinámica de la tasa de aprendizaje (ReduceLROnPlateau)

Se empleó *ReduceLROnPlateau* para ajustar automáticamente la tasa de aprendizaje cuando la mejora en validación se estancaba. Esta técnica ayuda al modelo a salir de mínimos locales y a refinar pesos de manera más precisa durante etapas avanzadas del entrenamiento.

6.5. Selección automática del mejor modelo (ModelCheckpoint)

Se utilizó *ModelCheckpoint* para guardar automáticamente la versión del modelo con el mejor desempeño en validación. Esto asegura que la arquitectura utilizada en la etapa de evaluación e inferencia sea la más efectiva observada durante todo el proceso de entrenamiento.

Gracias a la combinación de estas técnicas, el modelo logró mantener curvas de validación estables, sin señales significativas de sobreajuste y con mejoras progresivas tanto en accuracy como en loss. Estas estrategias fueron esenciales para obtener un desempeño sólido y un comportamiento generalizable en el conjunto de prueba.

7. Resultados y evaluación

La evaluación del modelo se realizó utilizando el conjunto de prueba original del dataset, compuesto por 3,000 imágenes distribuidas en las seis clases. Las métricas obtenidas permiten analizar tanto el desempeño global como el rendimiento por categoría.

7.1. Desempeño General en el Conjunto de Prueba

El modelo alcanzó una accuracy de entre **0.93 y 0.94**, dependiendo de la corrida final del entrenamiento. Esto indica que el sistema es capaz de clasificar correctamente alrededor del **93–94%** de las imágenes no vistas, lo cual es un resultado sólido considerando la variabilidad visual presente en el dataset.

7.2. Classification report

A continuación se muestra un resumen de las métricas obtenidas para cada clase, incluyendo **precision**, **recall**, **F1-score** y **support**:

Clase	Precision	Recall	F1-score	Support
Buildings	0.94	0.93	0.94	437
Forest	1.00	0.99	0.99	474
Glacier	0.90	0.87	0.89	553
Mountain	0.89	0.88	0.88	525
Sea	0.95	0.98	0.97	510
Street	0.94	0.95	0.94	501

Métricas globales:

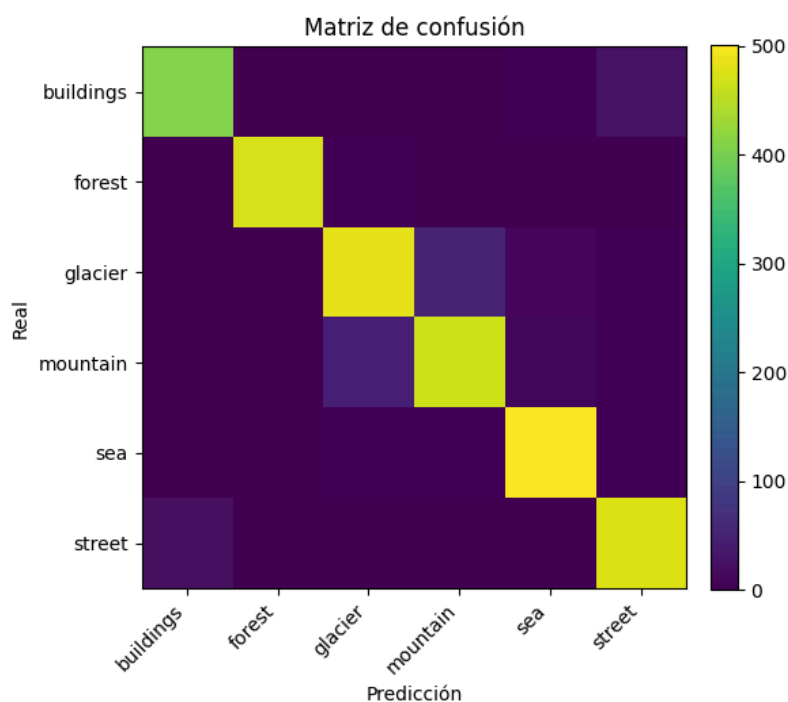
- **Accuracy:** 0.93
- **Macro avg:** 0.94
- **Weighted avg:** 0.93

7.2.1. Interpretación de los resultados

- Todas las clases alcanzan métricas **superiores al 88%** en F1-score.
- Las clases **forest**, **sea** y **buildings** presentan el desempeño más alto, destacando *forest* con un F1-score de **0.99**, lo que indica una separación visual muy clara en esas categorías.
- Las clases que presentan menor rendimiento relativo son **glacier** y **mountain**, aunque aún mantienen valores altos (≈ 0.88 – 0.89). Esto es esperable debido a su similitud visual como texturas, tonos fríos, presencia de hielo o nieve en ambos escenarios.
- El balance entre *precision* y *recall* en todas las clases indica que el modelo no solo predice bien, sino que además evita errores sistemáticos.

En general, el reporte refleja un modelo **bien equilibrado**, sin clases dominantes ni deterioro significativo en categorías específicas.

7.3 Matriz de confusión



7.3.1. Análisis de confusiones más relevantes

- **glacier ↔ mountain**

Hay confusiones notorias entre estas dos clases. Visualmente, ambas contienen elementos similares como nieve, hielo, montañas y terrenos fríos. Es un tipo de error esperable incluso en modelos avanzados.

- **buildings ↔ street**

Estas dos categorías también comparten similitudes contextuales como presencia de estructuras urbanas, cemento, calles, postes, etc. Esto provoca que ciertas imágenes fronterizas puedan confundirse.

- **forest, sea y street**

Muestran bloques prácticamente perfectos en la diagonal, lo que confirma una clara diferenciación visual de estas clases.

7.3.2. Conclusión del análisis de la matriz

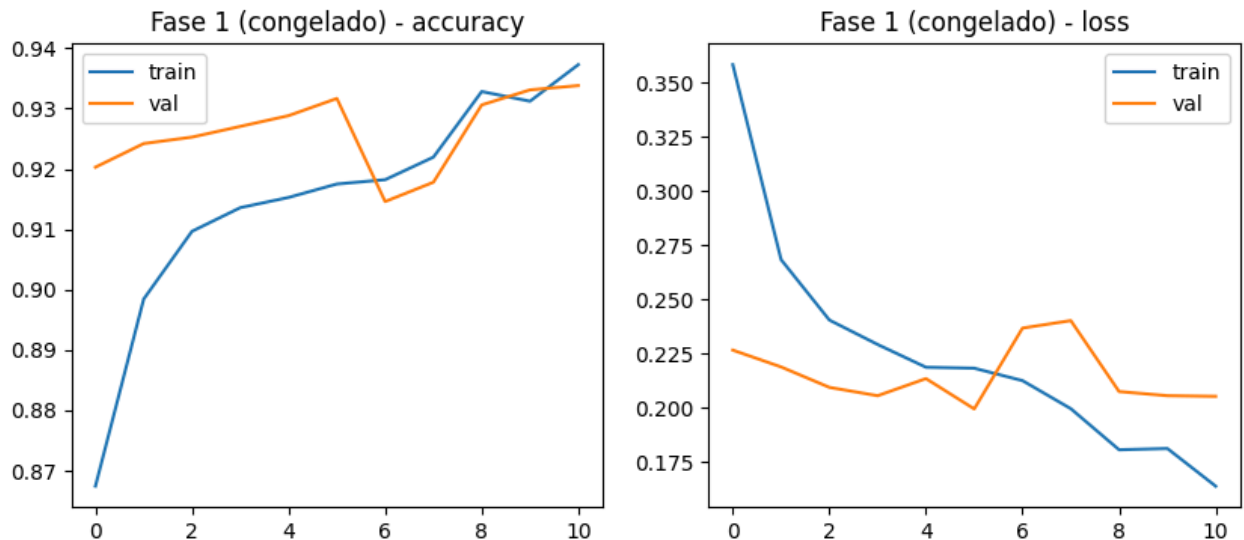
La mayoría de los errores ocurren en clases con características visuales similares. Aun así, el modelo logra mantener precisión alta en todas las categorías, demostrando una excelente capacidad de generalización.

8. Gráficas de entrenamiento

Para analizar el comportamiento del modelo durante las dos etapas de entrenamiento, se generaron gráficas comparativas de **accuracy** y **loss** para los conjuntos de entrenamiento y validación. Estas visualizaciones permiten identificar tendencias, estabilidad y posibles problemas como sobreajuste u oscilaciones en el aprendizaje.

8.1. Fase 1 – Modelo Congelado

Durante la primera etapa, donde únicamente se entrenaron las capas densas añadidas al modelo, se observa un comportamiento estable tanto en *accuracy* como en *loss*.



8.1.1. Accuracy

- La *accuracy* de entrenamiento aumenta de manera progresiva y consistente.
- La *accuracy* de validación sigue una tendencia similar, manteniéndose muy cercana a la de entrenamiento.
- No se observan separaciones abruptas entre las curvas, lo que indica buena alineación entre ambos conjuntos.

8.1.2. Loss

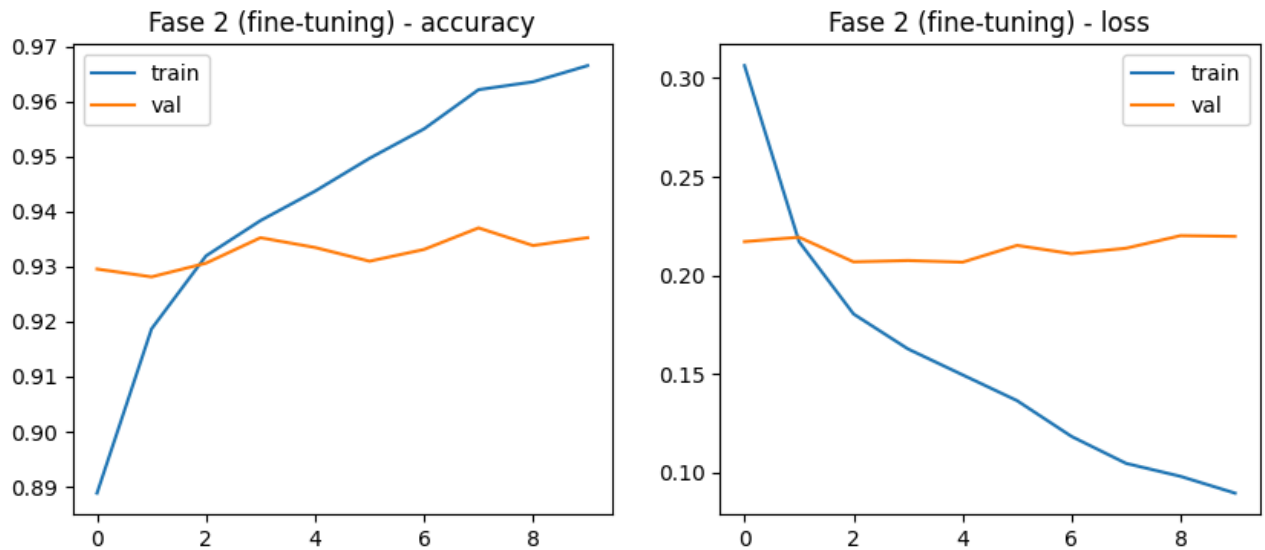
- La pérdida de entrenamiento disminuye de manera constante a lo largo de los epochs.
- La pérdida de validación también desciende, aunque presenta ligeras oscilaciones intermedias, algo normal al inicio del entrenamiento de las capas superiores.
- La distancia entre ambas curvas es reducida, lo que sugiere que el modelo aún no está sobreajustando.

8.1.3. Conclusión de la Fase 1

El modelo logra estabilizarse rápidamente, y las curvas muestran un aprendizaje controlado y adecuado para proceder a la etapa de fine-tuning.

8.2. Fase 2 – Fine-tuning del Modelo

En esta etapa, al descongelar las últimas capas de EfficientNetB0, el modelo continúa refinando patrones más profundos del dataset.



8.2.1. Accuracy

- La *accuracy* de entrenamiento sigue incrementando de manera sostenida, llegando a valores cercanos al 97%.
- La *accuracy* de validación se mantiene estable alrededor del 93–94%, sin caídas drásticas ni divergencias pronunciadas.
- Aunque la curva de entrenamiento es superior a la de validación, la diferencia es moderada y esperable en tareas de fine-tuning.

8.2.2. Loss

- La pérdida de entrenamiento disminuye significativamente durante los primeros epochs, reflejando una adaptación profunda del modelo.
- La pérdida de validación muestra oscilaciones suaves pero sin tendencia al alza, manteniéndose en un rango estable alrededor de 0.20.
- La distancia entre ambas curvas aumenta ligeramente, indicando un grado controlado de especialización sin caer en sobreajuste severo.

8.2.3. Conclusión de la Fase 2

El fine-tuning mejora la precisión general del modelo y estabiliza mejor la pérdida. Las curvas muestran un comportamiento saludable: el modelo aprende más, pero no se desvía de manera extrema respecto al conjunto de validación.

8.3. Conclusión General sobre las Gráficas

El análisis visual de *accuracy* y *loss* demuestra que:

- El modelo aprende de forma progresiva y equilibrada.
- No se identifican signos de sobreajuste grave.
- El fine-tuning aporta mejoras marginales pero significativas en estabilidad y rendimiento.
- Las métricas finales concuerdan con la evolución observada en las curvas.

Este comportamiento confirma que la arquitectura seleccionada, junto con las técnicas de regularización aplicadas, fue adecuada para resolver la tarea de clasificación de escenas con alto nivel de rendimiento.

9. Aplicación: Función para probar imágenes nuevas

Como parte final del proyecto, se implementó una función práctica que permite evaluar el modelo con imágenes externas seleccionadas por el usuario. Esta función replica exactamente el mismo flujo utilizado durante el entrenamiento, garantizando coherencia en el preprocesamiento y fiabilidad en las predicciones.

La función incluye los siguientes pasos:

• Carga de una imagen externa

El usuario puede subir cualquier archivo de imagen desde su dispositivo. Esto permite probar el modelo en escenarios reales y con imágenes fuera del dataset de entrenamiento.

• Preprocesamiento uniforme

La imagen se transforma al mismo tamaño utilizado por el modelo (**224×224**), se convierte a tensores y se normaliza mediante la función de preprocesamiento oficial de EfficientNet, asegurando compatibilidad total con las características aprendidas durante el entrenamiento.

• Predicción de clase y probabilidad

La imagen procesada se pasa al modelo, el cual genera un vector de probabilidades para las seis clases del dataset.

Se selecciona la clase con mayor probabilidad como la predicción principal.

• Visualización del resultado

La imagen se muestra con la clase predicha y el porcentaje de confianza, lo que facilita la interpretación del modelo desde una perspectiva visual.

• Probabilidades completas

Además de la predicción principal, se imprimen todas las probabilidades ordenadas de mayor a menor. Esto permite analizar el nivel de ambigüedad en casos donde la imagen pudiera corresponder a más de una categoría plausible.

Predicción principal: buildings (84.03%)



Probabilidades por clase:

buildings	: 84.03%
street	: 15.96%
sea	: 0.01%
glacier	: 0.00%
mountain	: 0.00%
forest	: 0.00%



9.1. Conclusión de la funcionalidad

La implementación de esta función demuestra la aplicabilidad real del modelo en contextos prácticos. Permite evaluar su rendimiento con imágenes totalmente nuevas, validar su capacidad de generalización y ofrecer una herramienta inmediata para pruebas rápidas por parte del usuario final o durante un despliegue inicial del sistema.

10. Conclusiones

El proyecto logró implementar con éxito un modelo robusto de clasificación de escenas en base a los conocimientos adquiridos dentro del módulo, aplicando de manera correcta los principios de preprocesamiento, arquitectura, entrenamiento y evaluación de modelos de deep learning. A través del uso de EfficientNetB0, transfer learning y técnicas de regularización, se desarrolló un pipeline completo que cubrió todas las etapas fundamentales del desarrollo de un modelo moderno: preparación del dataset, aumento de datos, construcción del modelo, entrenamiento en dos fases, callbacks avanzados, evaluación cuantitativa y cualitativa, y finalmente la inferencia con imágenes nuevas.

Como resultado de este proceso, el modelo alcanzó un accuracy en el conjunto de prueba entre 0.93 y 0.94, junto con métricas equilibradas en todas las clases, demostrando un desempeño estable y generalizable. Estas cifras reflejan no solo la eficacia del enfoque implementado, sino también la correcta aplicación de los conceptos revisados durante el módulo.

Finalmente, el sistema obtenido muestra un nivel de precisión y confiabilidad suficiente para integrarse en aplicaciones reales que requieran clasificación automática de imágenes, validando la importancia de las técnicas de deep learning estudiadas y la capacidad del modelo para resolver problemas visuales de manera eficiente y escalable.